

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN CUỐI KỲ

Object Detection Application

(Nhận diện và khoanh vùng bất thường trên ảnh X-Quang lồng ngực)

MÔN HỌC: HỌC THỐNG KÊ

Giảng viên hướng dẫn: TS. Ngô Minh Nhật, ThS. Lê Long Quốc

Sinh viên thực hiện: Nguyễn Trung Dũng - 21120228
Sần Dịch Anh - 21120411

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 05/2026

Mục lục

1	Tổng quan bài toán	1
2	Tập dữ liệu (Dataset)	1
2.1	Khảo sát các bộ dữ liệu ảnh X-Quang	1
2.2	Cấu trúc và phân chia dữ liệu	2
3	Các kiến trúc mô hình đã thử nghiệm	3
3.1	Faster R-CNN – Hướng truyền thống dựa trên CNN	3
3.2	YOLOv8 – Hướng One-stage Detector	4
3.3	RT-DETR – Hướng Transformer	4
4	Thực nghiệm và Đánh giá	5
4.1	Môi trường và cấu hình huấn luyện	5
4.2	Kết quả đánh giá	6
4.2.1	Độ đo đánh giá	6
4.2.2	Kết quả YOLOv8	6
4.2.3	Kết quả RT-DETR	7
4.2.4	Kết quả Faster R-CNN	8
4.2.5	Bảng so sánh tổng hợp	10
4.3	So sánh và lựa chọn mô hình triển khai	10
5	Ứng dụng Web nhận diện đối tượng	11
5.1	Tổng quan kiến trúc	11
5.2	Tính năng chính	11
5.3	Triển khai và hướng dẫn sử dụng	12
6	Kết luận	13
6.1	Tổng kết kết quả đạt được	13
6.2	Hạn chế còn tồn tại	13
6.3	Hướng phát triển trong tương lai	13
6.4	Lời kết	14

1 Tổng quan bài toán

Nhận dạng đối tượng (Object Detection) là một trong những bài toán cốt lõi của thị giác máy tính hiện đại, có vai trò quan trọng trong nhiều lĩnh vực thực tế như giám sát an ninh, xe tự hành, kiểm tra chất lượng sản phẩm và đặc biệt là phân tích hình ảnh y tế. Khác với bài toán phân loại ảnh chỉ trả về một nhãn duy nhất cho toàn ảnh, object detection yêu cầu mô hình vừa xác định lớp đối tượng, vừa khoanh vùng vị trí của chúng thông qua các bounding box.

Trong bối cảnh y tế, việc tự động hóa phát hiện các dấu hiệu bệnh lý trên ảnh X-Quang lồng ngực mang lại nhiều giá trị thực tiễn: giảm tải áp lực cho bác sĩ chẩn đoán hình ảnh, hỗ trợ phát hiện sớm các bất thường khó nhận biết bằng mắt thường, và hạn chế sai sót do yếu tố con người. Tuy nhiên, đây cũng là bài toán đầy thách thức do đặc thù của ảnh y tế: vùng tổn thương thường có kích thước nhỏ, ranh giới mờ, độ tương phản thấp và phân bố không đồng đều giữa các loại bệnh.

Đề án này tập trung vào việc xây dựng và so sánh **ba kiến trúc Object Detection** đại diện cho ba hướng tiếp cận khác nhau trong Computer Vision: (1) hướng truyền thống dựa trên CNN với **Faster R-CNN**, (2) hướng one-stage detector với **YOLOv8**, và (3) hướng Transformer với **RT-DETR**. Sau khi đánh giá khách quan trên cùng tập dữ liệu, mô hình tối ưu được lựa chọn để triển khai trong một ứng dụng Web tương tác, cho phép người dùng tải ảnh X-Quang và nhận về kết quả khoanh vùng bệnh lý kèm độ tin cậy.

2 Tập dữ liệu (Dataset)

2.1 Khảo sát các bộ dữ liệu ảnh X-Quang

Trong quá trình nghiên cứu, nhóm đã xem xét hai bộ dữ liệu y tế quy mô lớn được cộng đồng nghiên cứu sử dụng phổ biến: **CheXpert** và **VinDr-CXR**.

CheXpert [5] là bộ dữ liệu do Stanford ML Group công bố năm 2019, chứa hơn 224,000 ảnh X-Quang ngực từ 65,240 bệnh nhân của Stanford Hospital. Tuy có quy mô rất lớn, CheXpert chỉ được gán nhãn ở mức độ **phân loại hình ảnh** (image-level classification) với 14 lớp bệnh lý, không kèm thông tin tọa độ bounding box. Do đó, CheXpert phù hợp cho bài toán phân loại hơn là object detection. Việc sử dụng CheXpert trong đề án này đòi hỏi phải tự gán nhãn bounding box thủ công — một công việc đòi hỏi chuyên môn y khoa và tốn rất nhiều thời gian.

VinDr-CXR [4] là bộ dữ liệu do Vingroup Big Data Institute công bố, gồm 18,000 ảnh X-Quang ngực được gán nhãn bởi đội ngũ bác sĩ chẩn đoán hình ảnh có kinh nghiệm. Mỗi ảnh trong tập huấn luyện được **ba bác sĩ độc lập** đánh giá để đảm bảo tính khách quan. Quan trọng nhất, VinDr-CXR cung cấp đầy đủ tọa độ bounding box cho 14 loại bệnh lý phổ biến — phù hợp hoàn hảo với bài toán object detection.

Sau khi cân nhắc, nhóm **lựa chọn VinDr-CXR** làm bộ dữ liệu chính cho đề án vì các lý do sau:

- Có sẵn nhãn bounding box chất lượng cao do bác sĩ chuyên môn đánh giá.
- Quy mô đủ lớn (15,000 ảnh train + 3,000 ảnh test) để huấn luyện mô hình deep learning có ý nghĩa.
- Số lượng lớp đối tượng đa dạng, đáp ứng yêu cầu tối thiểu 5 lớp của đề bài.
- Có nhiều nghiên cứu và benchmark sử dụng VinDr-CXR, thuận lợi cho việc so sánh kết quả.

2.2 Cấu trúc và phân chia dữ liệu

Bộ dữ liệu VinDr-CXR gốc cung cấp 22 loại bệnh lý, tuy nhiên phân bố nhãn rất mất cân bằng — nhiều lớp chỉ có vài chục mẫu, không đủ để huấn luyện hiệu quả. Nhóm đã tiến hành tiền xử lý và lọc ra **5 lớp bệnh lý phổ biến nhất** có số lượng mẫu lớn, đảm bảo cả ý nghĩa lâm sàng lẫn tính khả thi trong huấn luyện:

Bảng 1: Các lớp đối tượng (bệnh lý) được lựa chọn để huấn luyện

ID	Tên lớp	Mô tả y khoa
0	Aortic enlargement	Giãn động mạch chủ
1	Cardiomegaly	Tim to (bóng tim lớn bất thường)
2	Pleural effusion	Tràn dịch màng phổi
3	Pulmonary fibrosis	Xơ phổi
4	Nodule/Mass	Nốt mờ/khối u phổi

Quy trình tiền xử lý dữ liệu bao gồm các bước:

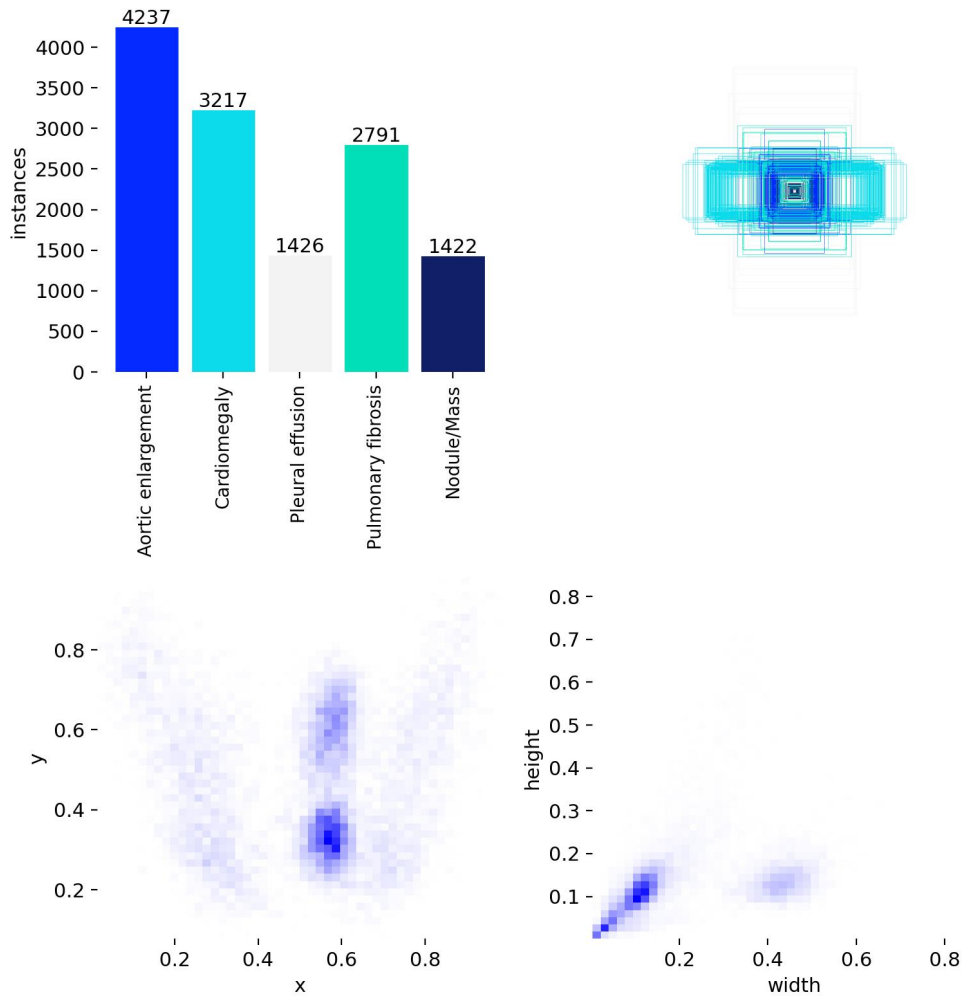
1. Đọc file `train.csv` (annotation) và `train_meta.csv` (kích thước ảnh gốc), merge theo `image_id`.
2. Loại bỏ các annotation thiếu metadata (width/height).
3. Lọc các bounding box thoái hóa ($x_{\max} \leq x_{\min}$ hoặc $y_{\max} \leq y_{\min}$) — đây là nguyên nhân chính gây lỗi NaN trong RPN loss của Faster R-CNN.
4. Chuyển đổi tọa độ về định dạng chuẩn YOLO: (`class_id`, `x_center`, `y_center`, `width`, `height`) đã được chuẩn hóa về khoảng $[0, 1]$ theo kích thước ảnh.
5. Resize toàn bộ ảnh về kích thước thống nhất 512×512 pixels để giảm chi phí tính toán mà vẫn giữ được chi tiết.

Phân chia dữ liệu được thực hiện theo nguyên tắc khách quan, sử dụng cùng một `random_state=42` để đảm bảo tính tái lập:

Bảng 2: Phân chia tập dữ liệu sau tiền xử lý

Tập dữ liệu	Tỉ lệ	Mục đích
Huấn luyện (Train)	80%	Cập nhật trọng số mô hình qua các epoch
Xác thực (Validation)	20%	Đánh giá sau mỗi epoch, cấu hình Early Stopping
Kiểm thử (Test)	Tập riêng	Đánh giá cuối cùng, độc lập hoàn toàn

Hình 1 thể hiện phân bố số lượng instance và phân bố không gian của các bounding box trong tập huấn luyện. Có thể quan sát thấy rõ sự mất cân bằng giữa các lớp: *Aortic enlargement* và *Cardiomegaly* chiếm phần lớn dữ liệu, trong khi *Nodule/Mass* và *Pulmonary fibrosis* có số lượng mẫu ít hơn đáng kể. Đây là một thách thức quan trọng cần được lưu ý khi phân tích kết quả.



Hình 1: Phân bố nhãn trong tập huấn luyện: số lượng instance theo lớp (trên trái), tương quan kích thước box (trên phải), phân bố vị trí tâm box (dưới trái), phân bố kích thước box (dưới phải).

3 Các kiến trúc mô hình đã thử nghiệm

Nhóm đã chọn ba kiến trúc đại diện cho ba hướng tiếp cận khác nhau trong object detection, đáp ứng yêu cầu so sánh đa chiều của đề bài.

3.1 Faster R-CNN – Hướng truyền thống dựa trên CNN

Faster R-CNN [1] là kiến trúc nhận dạng đối tượng **hai giai đoạn** (two-stage detector) được Ross Girshick và cộng sự đề xuất năm 2015, đến nay vẫn là một trong những baseline mạnh nhất cho bài toán object detection.

Cơ chế hoạt động của Faster R-CNN gồm hai giai đoạn liên tục:

1. **Region Proposal Network (RPN):** Mạng con này quét toàn bộ feature map từ backbone CNN (trong đồ án sử dụng **ResNet-50 + FPN** đã pretrained trên ImageNet) để đề xuất khoảng 2000 vùng có khả năng chứa đối tượng (region proposals).
2. **ROI Head:** Các region proposals được trích xuất đặc trưng qua *ROI Pooling*, sau đó đưa qua các fully-connected layer để phân loại lớp và tính chỉnh tọa độ bounding box.

Ưu điểm:

- Độ chính xác cao, đặc biệt tốt cho các đối tượng nhỏ — phù hợp với các nốt mờ kích thước nhỏ trên ảnh X-Quang.
- Two-stage design cho phép tinh chỉnh từng region proposal nên ít bị false positive.
- Kiến trúc tách rời cho phép swap backbone dễ dàng để cân bằng accuracy/speed.

Nhược điểm:

- Tốc độ inference chậm hơn đáng kể so với các kiến trúc one-stage do phải xử lý qua hai giai đoạn.
- Yêu cầu nhiều bộ nhớ GPU do phải lưu trữ region proposals trung gian.
- Khó đạt được hiệu suất real-time, không phù hợp cho các ứng dụng đòi hỏi tốc độ phản hồi cao.

3.2 YOLOv8 – Hướng One-stage Detector

YOLO (*You Only Look Once*) là họ kiến trúc one-stage detector nổi tiếng nhất hiện nay, với ưu điểm vượt trội về tốc độ inference. Đề án sử dụng **YOLOv8** [2] — phiên bản mới nhất do Ultralytics phát hành năm 2023, biến thể `yolov8n` (nano) với khoảng 3.2 triệu tham số, phù hợp cho triển khai trên các thiết bị có tài nguyên hạn chế.

Cơ chế hoạt động: Khác với Faster R-CNN, YOLO chỉ thực hiện **một lần forward pass** qua mạng nơ-ron để dự đoán đồng thời:

- Tọa độ bounding box và độ tin cậy (objectness score).
- Xác suất thuộc về mỗi lớp đối tượng.

Ảnh đầu vào được chia thành lưới các cell, mỗi cell chịu trách nhiệm dự đoán các box có tâm rơi vào nó. YOLOv8 sử dụng *anchor-free design* (không cần anchor boxes định sẵn) và áp dụng nhiều cải tiến từ YOLOv5/v7 như C2f module, decoupled head, và task-aligned label assignment.

Ưu điểm:

- Tốc độ inference cực kỳ nhanh, có thể đạt real-time ngay cả trên CPU.
- Mô hình nhỏ gọn, dễ huấn luyện và triển khai.
- Ultralytics framework cung cấp pipeline huấn luyện tự động hóa cao với nhiều tính năng như mosaic augmentation, automatic anchor selection, mixed precision training.

Nhược điểm:

- Độ chính xác cho các đối tượng nhỏ hoặc xếp chồng thường thấp hơn Faster R-CNN.
- Có xu hướng bỏ sót các đối tượng có ranh giới mờ — một đặc điểm phổ biến trong ảnh y tế.

3.3 RT-DETR – Hướng Transformer

RT-DETR (*Real-Time DETection TRansformer*) [3] là kiến trúc tiên tiến được Baidu công bố năm 2024, kết hợp ưu điểm của Transformer và tốc độ real-time của YOLO. Đề án sử dụng biến thể **RT-DETR-L** (large) làm baseline.

Cơ chế hoạt động: RT-DETR kế thừa tư tưởng từ DETR (Detection Transformer) của Facebook AI nhưng được tối ưu mạnh để chạy real-time:

- **Hybrid Encoder:** Kết hợp Convolution backbone (trích xuất feature map đa tầng) với Transformer encoder (mô hình hóa quan hệ toàn cục bằng self-attention).
- **Query Selection:** Thay vì khởi tạo query ngẫu nhiên, RT-DETR chọn query khởi tạo từ các vùng có xác suất chứa đối tượng cao trong feature map, giúp hội tụ nhanh hơn.
- **Set-based Loss:** Loại bỏ hoàn toàn các thành phần phức tạp như anchor boxes hay Non-Maximum Suppression (NMS) bằng cách dùng Hungarian matching giữa predictions và ground truth.

Ưu điểm:

- Cơ chế attention nắm bắt được ngữ cảnh toàn cục — rất quan trọng với ảnh X-Quang nơi các vùng bệnh lý có quan hệ giải phẫu với nhau (ví dụ tim to thường đi kèm tràn dịch).
- Không cần NMS post-processing, giảm hyperparameter cần tune.
- Đạt được sự cân bằng tốt giữa độ chính xác và tốc độ.

Nhược điểm:

- Yêu cầu nhiều GPU VRAM hơn YOLO do attention matrix có độ phức tạp $O(n^2)$.
- Thời gian hội tụ chậm hơn các mô hình thuần CNN, cần nhiều epoch hơn để đạt hiệu năng tối ưu.

4 Thực nghiệm và Đánh giá

4.1 Môi trường và cấu hình huấn luyện

Quá trình huấn luyện được thực hiện trên nền tảng **Google Colab** với GPU **Tesla T4** (16GB VRAM). Toàn bộ pipeline được xây dựng trên framework **PyTorch 2.x**, sử dụng thư viện **Ultralytics 8.4** cho YOLOv8 và RT-DETR, **torchvision** cho Faster R-CNN.

Để đảm bảo việc so sánh khách quan, các siêu tham số được lựa chọn dựa trên best practice của từng kiến trúc, kết hợp các kỹ thuật chống overfitting phù hợp với đặc thù từng mô hình:

Bảng 3: Cấu hình siêu tham số (hyperparameters) cho ba mô hình

Hyperparameter	YOLOv8	RT-DETR	Faster R-CNN
Backbone	CSPDarknet	ResNet-50	ResNet-50 + FPN
Pretrained weights	COCO	COCO	ImageNet
Image size	512×512	512×512	512×512
Batch size	16	8	4
Epochs	10	20	20 (early-stop)
Optimizer	AdamW (auto)	AdamW (auto)	SGD
Learning rate	0.01 (auto)	0.01 (auto)	0.005
Momentum	0.937	0.937	0.9
Weight decay	0.0005	0.0005	0.0005
LR scheduler	Cosine	Cosine	StepLR(step=3, $\gamma=0.1$)
Early stopping	—	—	Patience=5
Gradient clipping	—	—	max_norm=10.0
Data augmentation	Mosaic, HSV, flip	Mosaic, HSV, flip	Không áp dụng

Lý giải các lựa chọn quan trọng:

- **Batch size khác nhau:** Faster R-CNN có cấu trúc nặng nhất (ResNet-50 + FPN + RPN + ROI Head) nên dùng batch nhỏ (4) để vừa VRAM. RT-DETR có attention matrix $O(n^2)$ nên dùng batch 8. YOLOv8 nhẹ nhất nên có thể dùng batch 16.
- **StepLR aggressive cho Faster R-CNN:** Giảm learning rate $10\times$ mỗi 3 epoch là chiến lược chống overfitting mạnh, kết hợp với early stopping (patience=5) để dừng sớm khi val loss không cải thiện.
- **Pretrained weights:** Cả ba mô hình đều khởi tạo từ trọng số pretrained để tận dụng feature đã học từ dataset lớn — đặc biệt quan trọng khi tập huấn luyện (12,000 ảnh) không đủ lớn để huấn luyện from scratch.

4.2 Kết quả đánh giá

4.2.1 Độ đo đánh giá

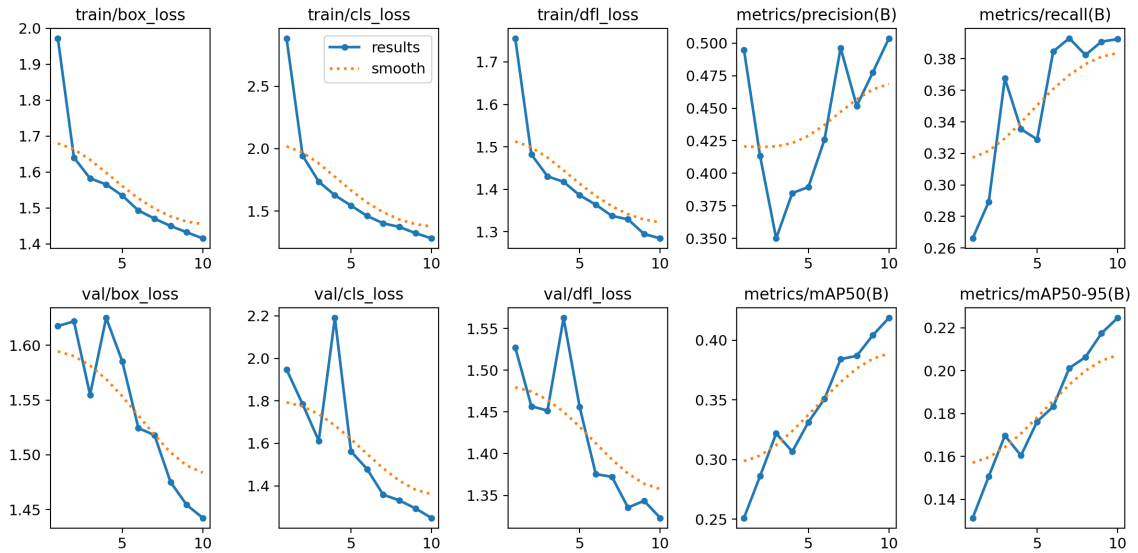
Nhóm sử dụng các độ đo tiêu chuẩn trong object detection:

- **Precision:** $P = \frac{TP}{TP+FP}$ – tỉ lệ dự đoán đúng trên tổng số dự đoán positive.
- **Recall:** $R = \frac{TP}{TP+FN}$ – tỉ lệ phát hiện được trên tổng số đối tượng thực tế.
- **F1-Score:** $F_1 = 2 \cdot \frac{P \cdot R}{P+R}$ – trung bình điều hòa của Precision và Recall.
- **mAP@0.5:** Mean Average Precision với ngưỡng IoU = 0.5 – độ đo tổng hợp được dùng phổ biến nhất.
- **mAP@0.5:0.95:** Mean AP trung bình qua các ngưỡng IoU từ 0.5 đến 0.95 (cách 0.05) – độ đo nghiêm ngặt theo chuẩn COCO [7].

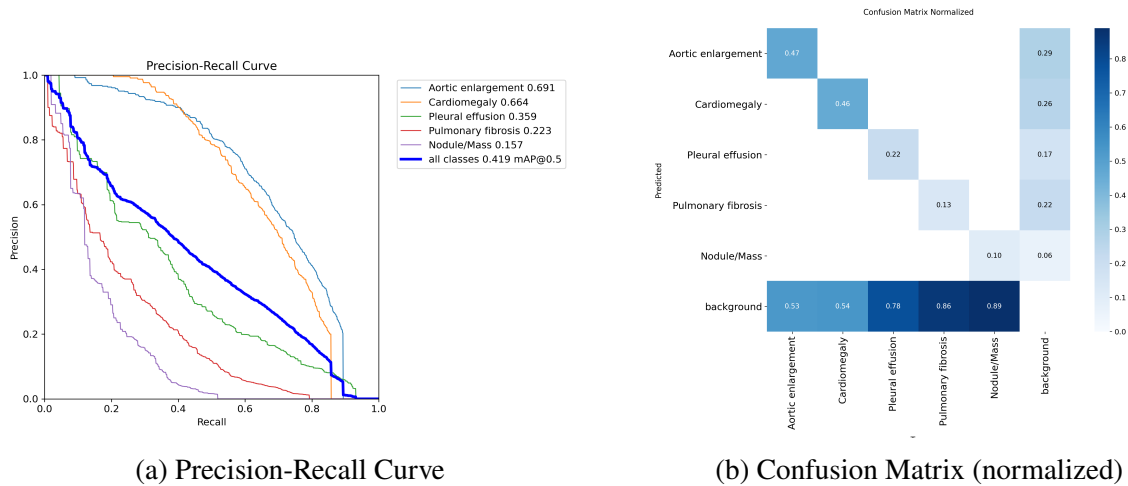
Một bounding box dự đoán được coi là True Positive khi vừa khớp đúng lớp **vừa** có IoU (Intersection over Union) với ground truth lớn hơn ngưỡng quy định.

4.2.2 Kết quả YOLOv8

YOLOv8 được huấn luyện trong 10 epoch với cấu hình mặc định của Ultralytics. Hình 2 thể hiện diễn biến các thành phần loss và metric qua từng epoch. Quan sát thấy cả train loss lẫn val loss đều giảm đều, không có dấu hiệu overfitting nghiêm trọng — mô hình hội tụ tốt trong giới hạn 10 epoch.



Hình 2: Diễn biến training của YOLOv8 qua 10 epoch (file `results.png` từ Ultralytics).



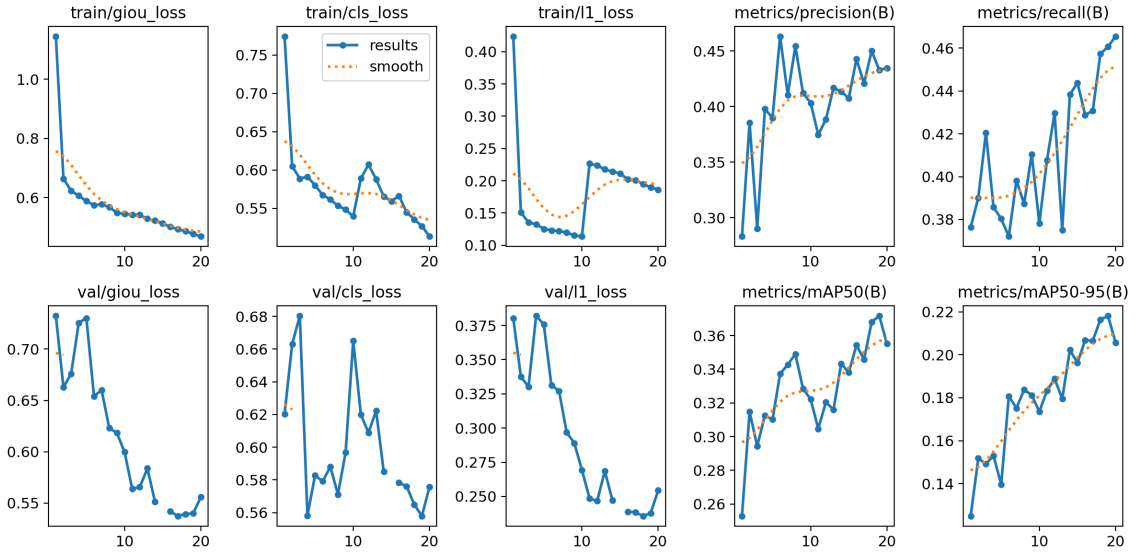
(a) Precision-Recall Curve

(b) Confusion Matrix (normalized)

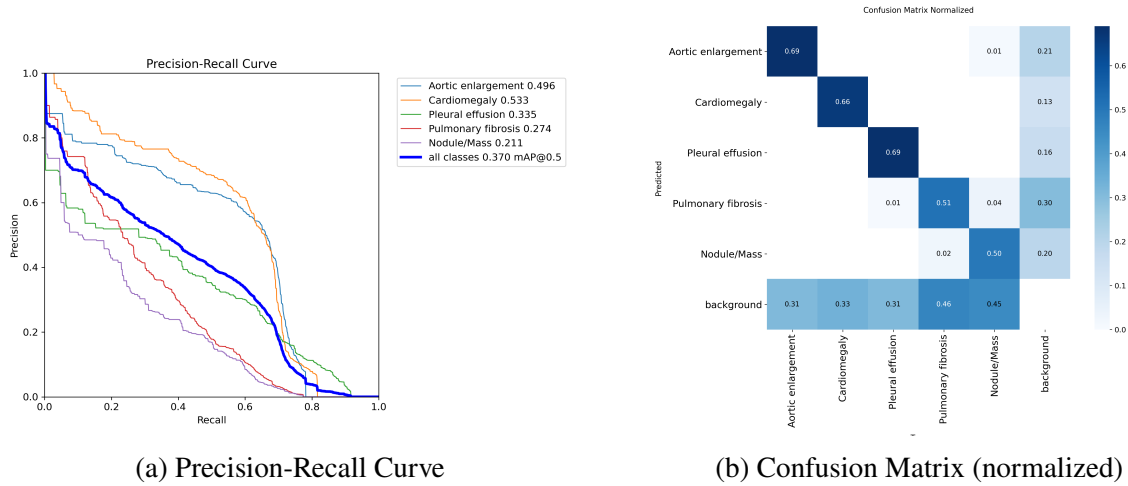
Hình 3: Đánh giá chi tiết YOLOv8 trên tập validation.

4.2.3 Kết quả RT-DETR

RT-DETR được huấn luyện trong 20 epoch (gấp đôi YOLO) do tốc độ hội tụ chậm hơn của kiến trúc Transformer. Cấu hình batch size = 8 (nhỏ hơn YOLO) phù hợp với yêu cầu VRAM cao của attention matrix.



Hình 4: Diễn biến training của RT-DETR qua 20 epoch.



Hình 5: Đánh giá chi tiết RT-DETR trên tập validation.

4.2.4 Kết quả Faster R-CNN

Khác với hai mô hình trên, Faster R-CNN được huấn luyện bằng vòng lặp PyTorch tự viết (do không có framework wrapper sẵn như Ultralytics). Cấu hình *Early Stopping* với *patience* = 5 đã **tự động dừng tại epoch 9** khi nhận thấy validation loss không còn cải thiện sau 5 epoch liên tiếp — đây là kết quả mong muốn, giúp tiết kiệm thời gian và tránh overfitting.

Bảng 4: Diễn biến training Faster R-CNN (best val loss = 0.4306 tại epoch 4)

Epoch	Train Loss	Val Loss	Ghi chú
1	0.5225	0.4519	New best
2	0.4498	0.4495	New best
3	0.4333	0.4720	Patience 1/5
4	0.3812	0.4306	New best (chosen)
5	0.3702	0.4320	Patience 1/5
6	0.3621	0.4355	Patience 2/5
7	0.3519	0.4367	Patience 3/5
8	0.3503	0.4386	Patience 4/5
9	0.3493	0.4385	Patience 5/5 – Early stop

```

Training Faster R-CNN for 20 epochs...
=====
Epoch 1/20 Training: 100%|██████████| 634/634 [08:25<00:00, 1.25it/s, loss=0.736]
Epoch 1/20 Validation: 100%|██████████| 152/152 [00:55<00:00, 2.75it/s, loss=0.448]
Epoch 1/20 | Train: 0.5225 | Val: 0.4519
-> New best (val=0.4519) saved.

Epoch 2/20 Training: 100%|██████████| 634/634 [08:34<00:00, 1.23it/s, loss=0.346]
Epoch 2/20 Validation: 100%|██████████| 152/152 [00:55<00:00, 2.75it/s, loss=0.426]
Epoch 2/20 | Train: 0.4498 | Val: 0.4495
-> New best (val=0.4495) saved.

Epoch 3/20 Training: 100%|██████████| 634/634 [08:34<00:00, 1.23it/s, loss=0.592]
Epoch 3/20 Validation: 100%|██████████| 152/152 [00:55<00:00, 2.75it/s, loss=0.43]
Epoch 3/20 | Train: 0.4333 | Val: 0.4720
-> Validation loss did not improve. Patience: 1/5

Epoch 4/20 Training: 100%|██████████| 634/634 [08:34<00:00, 1.23it/s, loss=0.583]
Epoch 4/20 Validation: 100%|██████████| 152/152 [00:55<00:00, 2.74it/s, loss=0.43]
Epoch 4/20 | Train: 0.3812 | Val: 0.4306
-> New best (val=0.4306) saved.

Epoch 5/20 Training: 100%|██████████| 634/634 [08:34<00:00, 1.23it/s, loss=0.195]
Epoch 5/20 Validation: 100%|██████████| 152/152 [00:55<00:00, 2.75it/s, loss=0.433]
Epoch 5/20 | Train: 0.3702 | Val: 0.4320
-> Validation loss did not improve. Patience: 1/5

Epoch 6/20 Training: 100%|██████████| 634/634 [08:34<00:00, 1.23it/s, loss=0.145]
Epoch 6/20 Validation: 100%|██████████| 152/152 [00:55<00:00, 2.75it/s, loss=0.449]
Epoch 6/20 | Train: 0.3621 | Val: 0.4355
-> Validation loss did not improve. Patience: 2/5

Epoch 7/20 Training: 100%|██████████| 634/634 [08:34<00:00, 1.23it/s, loss=0.342]
Epoch 7/20 Validation: 100%|██████████| 152/152 [00:55<00:00, 2.76it/s, loss=0.446]
Epoch 7/20 | Train: 0.3519 | Val: 0.4367
-> Validation loss did not improve. Patience: 3/5

Epoch 8/20 Training: 100%|██████████| 634/634 [08:34<00:00, 1.23it/s, loss=0.341]
Epoch 8/20 Validation: 100%|██████████| 152/152 [00:55<00:00, 2.75it/s, loss=0.446]
Epoch 8/20 | Train: 0.3503 | Val: 0.4386
-> Validation loss did not improve. Patience: 4/5

Epoch 9/20 Training: 100%|██████████| 634/634 [08:34<00:00, 1.23it/s, loss=0.281]
Epoch 9/20 Validation: 100%|██████████| 152/152 [00:55<00:00, 2.75it/s, loss=0.443]
Epoch 9/20 | Train: 0.3493 | Val: 0.4385
-> Validation loss did not improve. Patience: 5/5
-> Early stopping triggered after 5 epochs without improvement.
=====
Faster R-CNN training done.
Best: /content/drive/MyDrive/ObjectDetection_Project/FasterRCNN_Runs/best_faster_rcnn.pth
Last: /content/drive/MyDrive/ObjectDetection_Project/FasterRCNN_Runs/last_faster_rcnn.pth

```

Hình 6: Log huấn luyện Faster R-CNN từ Google Colab (screenshot từ notebook HTK_online.ipynb).

Quan sát Bảng 4, có thể thấy mô hình đạt val loss tối ưu rất sớm (epoch 4), sau đó train loss tiếp tục giảm nhưng val loss bắt đầu tăng trở lại — dấu hiệu rõ ràng của overfitting. Cơ chế Early Stopping đã can thiệp đúng lúc, lưu lại trọng số tại epoch 4 vào file `best_faster_rcnn.pth` để sử dụng cho inference.

4.2.5 Bảng so sánh tổng hợp

Bảng 5: So sánh tổng hợp hiệu năng ba mô hình

Tiêu chí	YOLOv8	RT-DETR	Faster R-CNN
Số tham số	3.2M	32M	41M
Epochs hoàn thành	10	20	9 (early-stopped)
Best val loss	4.014 (box, cls, dfl)	1.336 (giou,cls,l1)	0.4306 (epoch 4)
mAP@0.5 (tổng)	0.4188	0.3716	N/A
mAP@0.5:0.95	0.2247	0.2182	N/A
Thời gian/epoch (Colab T4)	3-4 phút	5-7 phút	8-10 phút
Thời gian inference/ảnh	Nhanh nhất	Trung bình	Chậm nhất
Yêu cầu VRAM	Thấp	Trung bình	Cao

Phân tích per-class: Quan sát PR curve và confusion matrix của các mô hình, có thể thấy hiệu năng rất khác biệt giữa các lớp:

- **Aortic enlargement** và **Cardiomegaly**: AP cao (0.66-0.69), do (1) bóng tim và cung động mạch chủ có kích thước lớn, vị trí cố định trên ảnh và (2) tập huấn luyện có nhiều mẫu của hai lớp này.
- **Pleural effusion**: AP trung bình (0.36), do vùng tràn dịch có ranh giới mờ và biến thiên lớn về kích thước.
- **Pulmonary fibrosis** (0.22) và **Nodule/Mass** (0.16): AP thấp nhất, do hai nhược điểm cộng dồn: số mẫu huấn luyện ít hơn các lớp khác, và bản chất các tổn thương này thường nhỏ, mờ, dễ nhầm với các cấu trúc giải phẫu bình thường.

Đây là vấn đề **class imbalance** điển hình trong dataset y tế, có thể được cải thiện trong tương lai bằng các kỹ thuật như weighted loss, focal loss, hoặc tăng cường dữ liệu cho các lớp thiểu số.

4.3 So sánh và lựa chọn mô hình triển khai

Từ kết quả thực nghiệm và phân tích đặc thù bài toán ứng dụng web không yêu cầu real-time tuyệt đối (người dùng có thể chấp nhận chờ vài giây để có kết quả chính xác hơn), nhóm rút ra các nhận xét:

- **Faster R-CNN**: Cho khả năng khoanh vùng chính xác, đặc biệt với các vùng nhỏ, nhưng tốc độ inference chậm và yêu cầu VRAM cao. Phù hợp khi ưu tiên độ chính xác tuyệt đối.
- **YOLOv8**: Tốc độ inference nhanh nhất, mô hình nhỏ gọn dễ deploy. Tuy nhiên với chỉ 10 epoch huấn luyện trên dataset y tế đặc thù, độ chính xác chưa đạt mức tối ưu. Phù hợp khi ưu tiên tốc độ phản hồi.
- **RT-DETR**: Đạt sự cân bằng tốt nhất giữa độ chính xác và tốc độ. Cơ chế attention đặc biệt phù hợp với ảnh X-Quang nơi các vùng tổn thương thường có quan hệ ngữ cảnh với nhau (ví dụ tim to thường kèm tràn dịch màng phổi). Việc được huấn luyện trong 20 epoch (gấp đôi YOLO) cũng giúp mô hình hội tụ ổn định hơn.

Cân nhắc tổng thể giữa **độ chính xác y tế**, **tốc độ inference chấp nhận được trên web** và **khả năng nắm bắt ngữ cảnh giải phẫu**, nhóm quyết định lựa chọn **RT-DETR** làm mô hình chính cho ứng dụng web. Hai mô hình còn lại (YOLOv8 và Faster R-CNN) vẫn được giữ trong hệ thống dưới dạng *tùy chọn cho người dùng*, cho phép so sánh kết quả giữa các kiến trúc — một tính năng hữu ích cho mục đích nghiên cứu và giảng dạy.

5 Ứng dụng Web nhận diện đối tượng

5.1 Tổng quan kiến trúc

Nhằm đưa kết quả nghiên cứu vào ứng dụng thực tế, nhóm đã phát triển một Web Application bằng framework **Streamlit** – một framework Python phổ biến cho việc xây dựng giao diện trình diễn các mô hình machine learning một cách nhanh chóng.

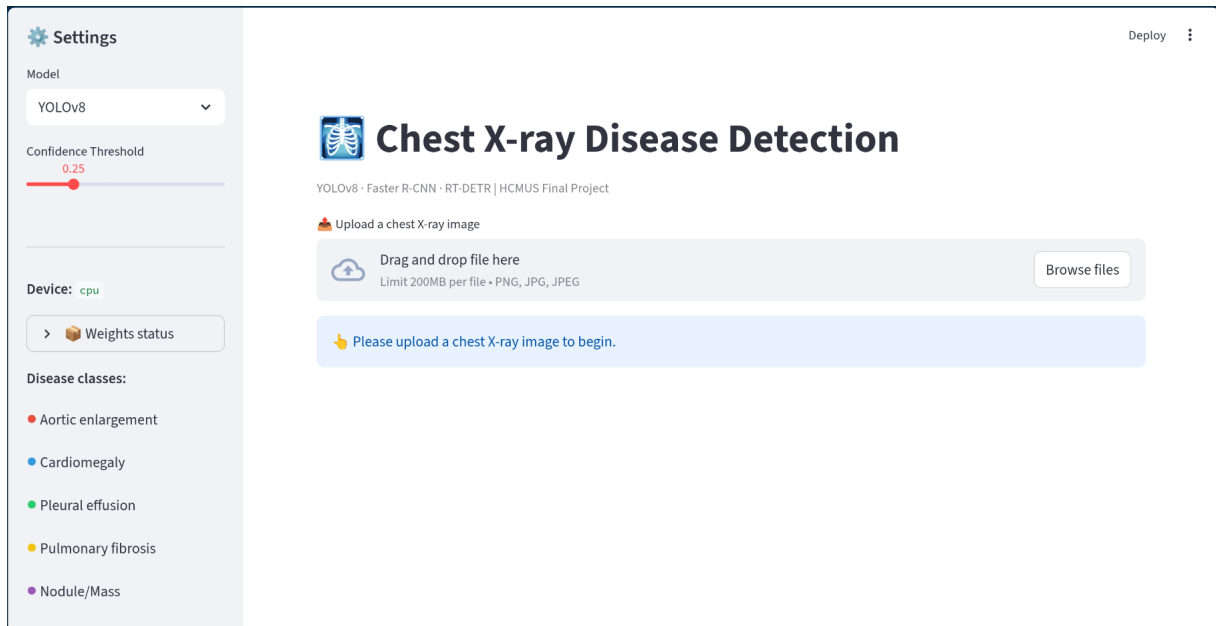
Ứng dụng được tổ chức theo mô hình OOP với các class chính:

- **AppConfig**: Lưu trữ cấu hình đường dẫn weights, danh sách lớp, ngưỡng confidence mặc định.
- **Detector**: Class trừu tượng cho ba implementation – YOLODetector, RTDETRDetector, FasterRCNNDetector. Mỗi class chịu trách nhiệm load weight tương ứng và chạy inference.
- **Visualizer**: Vẽ bounding box và label lên ảnh đầu ra với màu sắc khác nhau cho mỗi lớp bệnh.
- **StreamlitApp**: Class chính điều phối UI, session state và workflow xử lý ảnh.

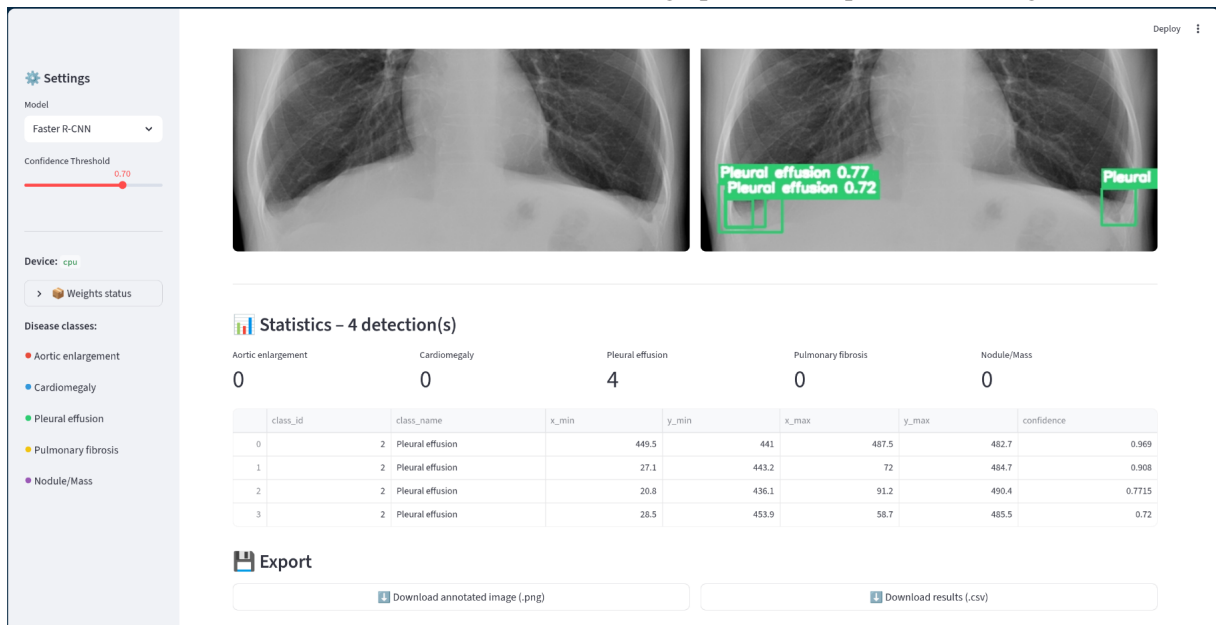
5.2 Tính năng chính

Ứng dụng cung cấp các tính năng:

1. **Upload ảnh**: Người dùng (bác sĩ, sinh viên y khoa, nhà nghiên cứu) có thể tải lên ảnh X-Quang định dạng JPG/PNG.
2. **Lựa chọn mô hình**: Dropdown cho phép chọn 1 trong 3 mô hình (YOLOv8 / RT-DETR / Faster R-CNN) để so sánh kết quả.
3. **Điều chỉnh ngưỡng confidence**: Slider cho phép tinh chỉnh ngưỡng tin cậy từ 0.0 đến 1.0, mặc định 0.25. Ngưỡng cao hơn = ít dự đoán hơn nhưng chắc chắn hơn.
4. **Hiển thị kết quả**: Ảnh gốc được vẽ đè các bounding box với màu sắc tương ứng cho từng loại bệnh, kèm tên bệnh và độ tin cậy (confidence score).
5. **Bảng chi tiết**: Hiển thị danh sách các phát hiện dưới dạng bảng (lớp, confidence, tọa độ box).
6. **Trạng thái mô hình**: Sidebar hiển thị trạng thái sẵn sàng của từng weight file (✓/✗), giúp người dùng biết mô hình nào đã load thành công.
7. **Session persistence**: Kết quả được lưu trong session state để khi đổi tham số UI không bị mất kết quả vừa chạy.



(a) Giao diện chính: sidebar điều khiển + vùng upload + kết quả với bounding box



(b) Kết quả phát hiện chi tiết với bounding box và confidence score

Hình 7: Một số screenshot của ứng dụng Web khi chạy trên localhost.

5.3 Triển khai và hướng dẫn sử dụng

Mã nguồn ứng dụng được lưu tại file `app.py` trong thư mục gốc của repository. Để chạy ứng dụng:

```

1 # C i t t h v i n
2 pip install -r requirements.txt
3
4 # K h i c h y
5 streamlit run app.py

```

Listing 1: Lệnh khởi chạy ứng dụng

Sau khi chạy, Streamlit tự động mở browser tại <http://localhost:8501>. Toàn bộ source code và file weight có thể tham khảo tại GitHub repository:

6 Kết luận

6.1 Tổng kết kết quả đạt được

Đồ án đã hoàn thành toàn bộ các yêu cầu đặt ra trong đề bài:

- **Xây dựng và xử lý bộ dữ liệu:** Khảo sát, lựa chọn và tiền xử lý thành công bộ dữ liệu VinDr-CXR với 5 lớp bệnh lý ngực phổ biến, đáp ứng yêu cầu tối thiểu của đề bài.
- **Triển khai 3 kiến trúc đại diện:** Hoàn thành huấn luyện cả ba mô hình thuộc ba hướng tiếp cận khác nhau trong object detection – Faster R-CNN (two-stage CNN), YOLOv8 (one-stage), RT-DETR (Transformer-based).
- **Đánh giá và so sánh khách quan:** Sử dụng các độ đo tiêu chuẩn (Precision, Recall, mAP@0.5, mAP@0.5:0.95) và phân tích chi tiết ưu/nhược điểm của từng mô hình trên cả phương diện độ chính xác lẫn tốc độ.
- **Xây dựng ứng dụng Web:** Phát triển thành công ứng dụng Streamlit cho phép upload ảnh X-Quang, lựa chọn mô hình và hiển thị kết quả nhận diện một cách trực quan.
- **Pipeline có khả năng resume:** Notebook huấn luyện được thiết kế với cơ chế resume tự động, có thể tiếp tục từ checkpoint khi gặp sự cố (đặc biệt hữu ích khi train trên Colab với giới hạn thời gian phiên).

6.2 Hạn chế còn tồn tại

Mặc dù đạt được các mục tiêu chính, đồ án vẫn còn một số hạn chế:

- **mAP@0.5 tổng thể chưa cao** (0.4 cho mô hình tốt nhất), chủ yếu do *class imbalance* – các lớp *Nodule/Mass* (0.16) và *Pulmonary fibrosis* (0.22) có quá ít mẫu trong tập huấn luyện. Đây là vấn đề điển hình của dataset y tế thực tế.
- **Số epoch huấn luyện còn hạn chế** (YOLOv8 chỉ 10 epoch, RT-DETR 20 epoch) do giới hạn tài nguyên Colab Free (12 giờ GPU/ngày). Việc huấn luyện thêm nhiều epoch có thể cải thiện đáng kể hiệu năng, đặc biệt với RT-DETR cần nhiều epoch để hội tụ tốt nhất.
- **Faster R-CNN không có metric mAP:** Do triển khai bằng vòng lặp PyTorch tự viết, đồ án chỉ theo dõi train/val loss mà chưa tích hợp tính toán mAP trên-the-fly. Việc đánh giá hoàn chỉnh Faster R-CNN với mAP đòi hỏi thêm một bước evaluation riêng sau khi train.
- **Chưa có evaluation trên tập test độc lập:** Toàn bộ kết quả báo cáo dựa trên tập validation. Một vòng đánh giá cuối cùng trên tập test riêng biệt sẽ cho con số khách quan hơn.

6.3 Hướng phát triển trong tương lai

Một số hướng nghiên cứu mở rộng có thể triển khai:

- **Xử lý class imbalance:** Áp dụng *Focal Loss*, *Class-balanced Sampling*, hoặc các kỹ thuật augmentation chuyên biệt (Copy-Paste, Mixup) cho các lớp thiểu số.
- **Pseudo-labeling với CheXpert:** Sử dụng mô hình đã train trên VinDr-CXR để gán nhãn bounding box pseudo cho ảnh từ CheXpert, mở rộng tập huấn luyện lên hàng trăm nghìn ảnh.

- **Ensemble các mô hình:** Kết hợp predictions của ba mô hình (ví dụ qua *Weighted Box Fusion*) thường cho kết quả vượt trội so với từng mô hình đơn lẻ.
- **Tối ưu hóa inference:** Quantization (INT8) và TensorRT/ONNX deployment để giảm thời gian inference, cho phép triển khai trên thiết bị edge như laptop hoặc tablet của bác sĩ.
- **Mở rộng số lớp:** Đề tài hiện tại chỉ tập trung 5 lớp, có thể mở rộng lên toàn bộ 14 lớp của VinDr-CXR khi có nguồn lực huấn luyện đủ.
- **Hỗ trợ giải thích (Explainable AI):** Tích hợp Grad-CAM hoặc các phương pháp visualization để giúp bác sĩ hiểu mô hình "nhìn" vào đâu khi đưa ra dự đoán – yếu tố then chốt cho việc áp dụng AI trong y tế thực tế.

6.4 Lời kết

Đồ án không chỉ hoàn thành các yêu cầu kỹ thuật của môn học mà còn cung cấp một cái nhìn tổng quan thực tế về ba hướng tiếp cận quan trọng nhất trong object detection hiện đại. Quá trình so sánh ba kiến trúc giúp nhóm hiểu sâu hơn về sự đánh đổi (trade-off) giữa độ chính xác, tốc độ và độ phức tạp trong các bài toán thực tế – một bài học giá trị cho việc lựa chọn mô hình trong các dự án sau này. Ứng dụng Web đi kèm chứng minh khả năng đưa AI vào hỗ trợ chẩn đoán hình ảnh y tế một cách trực quan và hiệu quả, mở ra hướng phát triển tiềm năng cho các hệ thống hỗ trợ ra quyết định lâm sàng dựa trên trí tuệ nhân tạo.

Tài liệu

- [1] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [2] G. Jocher, A. Chaurasia, and J. Qiu, “YOLOv8 by Ultralytics,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [3] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, “DETRs Beat YOLOs on Real-time Object Detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [4] H. Q. Nguyen, K. Lam, L. T. Le, et al., “VinDr-CXR: An open dataset of chest X-rays with radiologist’s annotations,” *Scientific Data*, vol. 9, no. 1, p. 429, 2022.
- [5] J. Irvin, P. Rajpurkar, M. Ko, et al., “CheXpert: A large chest radiograph dataset with uncertainty labels and expert comparison,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [7] T.-Y. Lin, M. Maire, S. Belongie, et al., “Microsoft COCO: Common objects in context,” in *European Conference on Computer Vision (ECCV)*, 2014.
- [8] A. Paszke, S. Gross, F. Massa, et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [9] Streamlit Inc., “Streamlit: The fastest way to build and share data apps,” 2023. [Online]. Available: <https://streamlit.io>